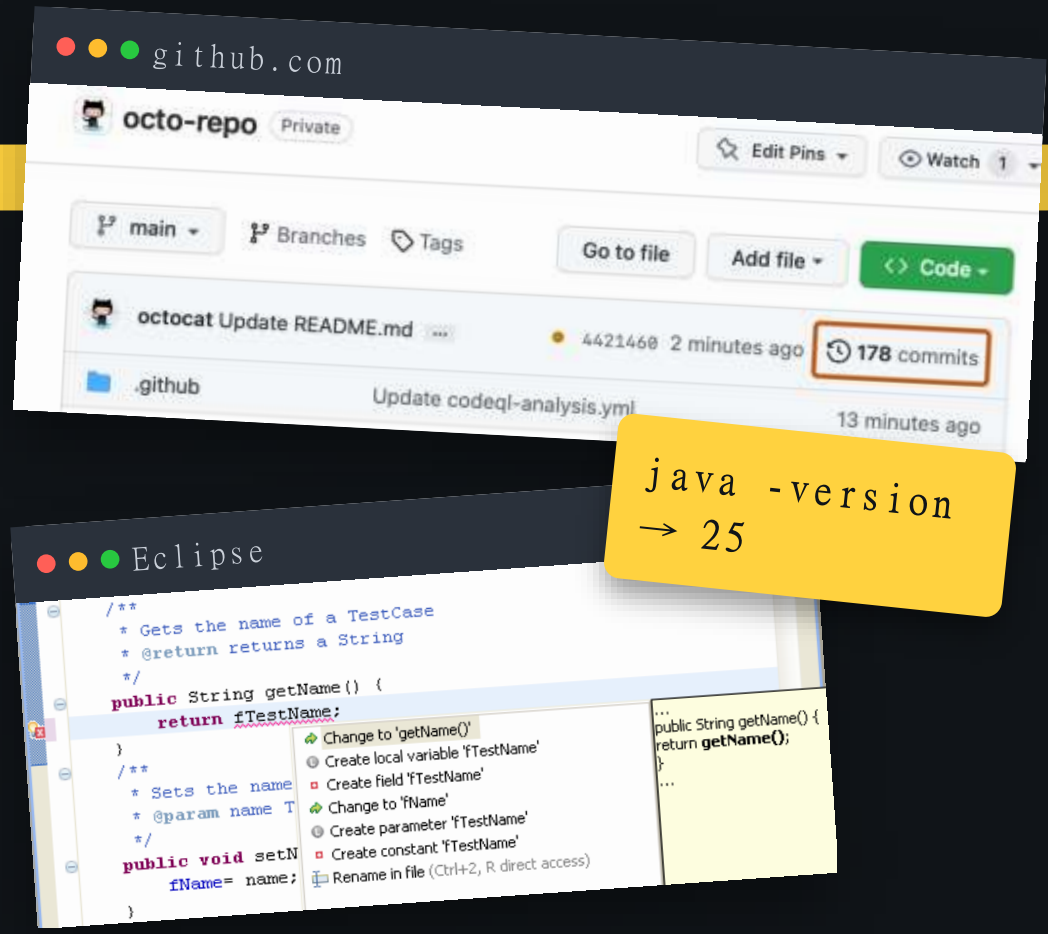
Seminar 1 · OBJECT-ORIENTED PROGRAMMING

Java, Eclipse & GitHub

Today you set up the three tools you will use every week, and push your first commit.



Alba Meca

TODAY

Four stops, one goal: your first push

01

The course & OOP

What we'll build and how we'll think

02

Java

Install it, compile & run

03

Eclipse

Install it, run a class

04

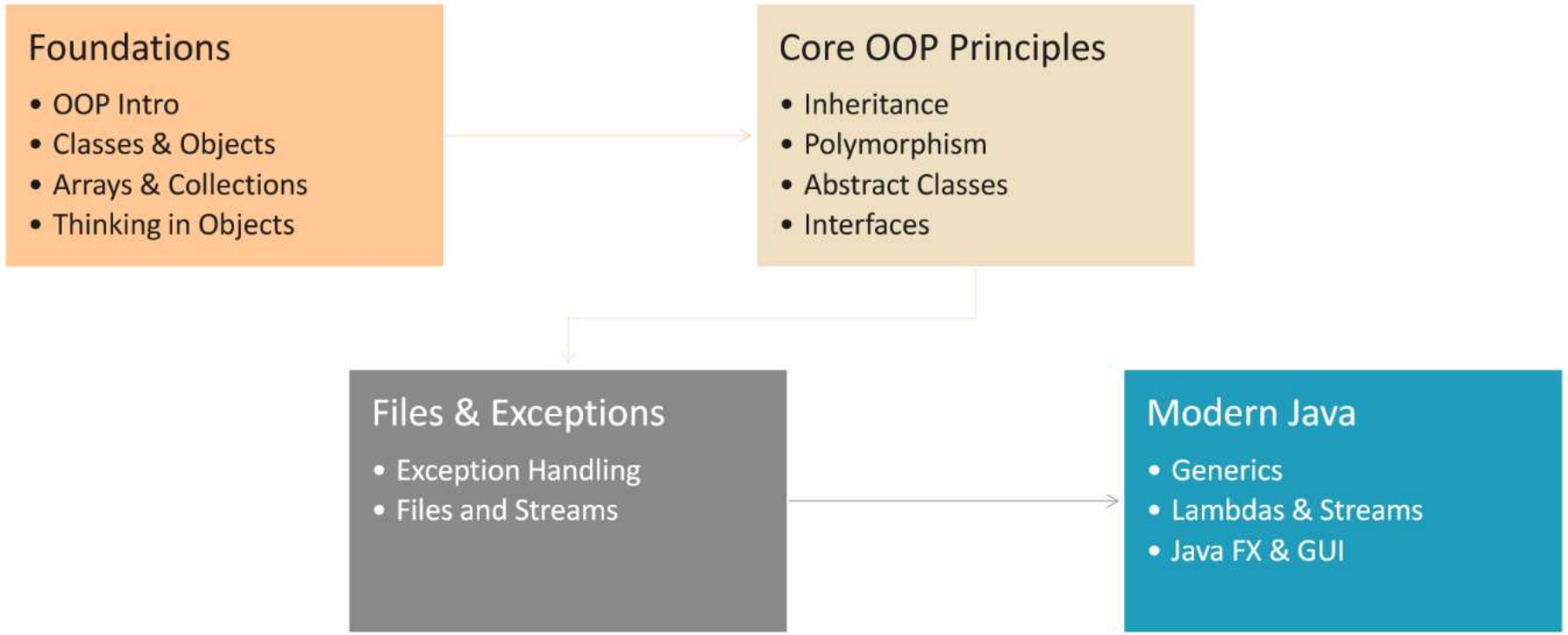
Git & GitHub

Your repo, your first push

THE COURSE



Basics first, core ideas next, then modern Java



THE COURSE

Following an example teaches syntax. Building teaches the skill.

LECTURE · 2 HOURS

Theory and live coding. One example grows all semester: **a small bank**.

SEMINAR · 2 HOURS · TODAY

You write the code, on a different problem: **the university system**. One folder, one repository, extended every week.



Nothing restarts from zero. By week 4 your seminar folder becomes the starting point for **Sprint 1**.

The only way to learn a programming language is by coding.



This is the way!

Hands up if you have ever...

used an OOP language

used Java

pushed code to GitHub

lost work you really needed

TODAY

Three tools, in this order



Install Java **before** Eclipse. Most setup problems come from doing it the other way round.

You are done when:

- **java -version** reports 25
- **javac -version** reports 25
- Eclipse opens and runs a class
- Your repo exists on GitHub with a commit you pushed

Not in OneDrive, Desktop or Documents. Cloud sync rewrites files while the compiler reads them. Use C:\ dev\

PART 1 OF 4

Object-oriented languages

Programs built from objects that hold data and behaviour together

“I thought of objects being like biological cells and/or individual computers on a network, only able to communicate with messages […].”

Dr. Alan Kay

Dr. Alan Kay on the Meaning of “Object-Oriented Programming”, 2003. Retrieved 11 February 2010

01 · OBJECT-ORIENTED LANGUAGES

One blueprint, many objects

CLASS = the blueprint

Student.java

```java
public class Student {
    String name;
    double gpa;
    void updateGpa(...) { ... }
}

Student ana = new Student(...);
```

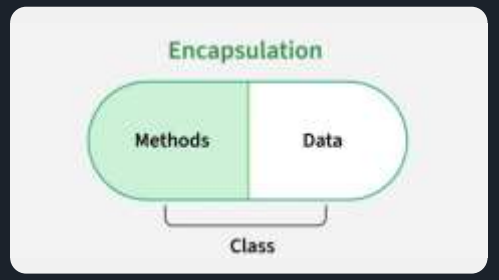
OBJECTS = the real things

ana : Student
name = "Ana" gpa = 3.6

erion : Student
name = "Erion" gpa = 3.1

sara : Student
name = "Sara" gpa = 3.9

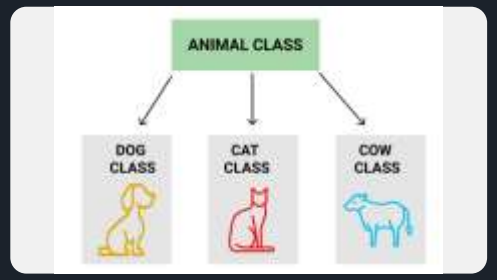
The four pillars of OOP



Encapsulation

Keep data and the methods that guard it together, and hide the data.

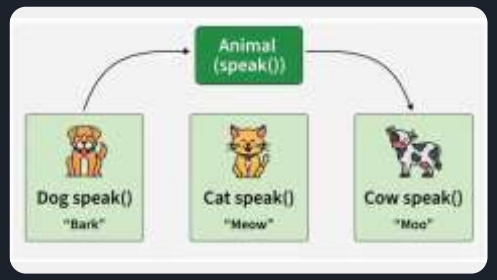
```
private double balance;
public void deposit(...)
```



Inheritance

A class reuses and extends another: a Dog is an Animal.

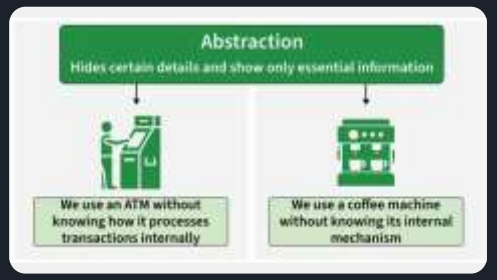
```
class Dog extends Animal
```



Polymorphism

One call, different behaviour: each animal speaks its own way.

```
animal.speak();
// Bark? Meow? Moo?
```



Abstraction

Use something by what it does, not how: an ATM, a coffee machine.

```
atm.withdraw(50);
// how? hidden from you
```

4 Pillars of OOP

Encapsulation



Polymorphism



Inheritance



Abstraction



01 · OBJECT-ORIENTED LANGUAGES

OOP is an idea. Many languages use it.

1972
Smalltalk
Pure OO

1985
C++
OO added to C

1991
Python
Data, scripts, AI

1995
Java
Class-based, statically typed. Enterprise, Android, back-ends.
THIS COURSE

2000
C#
.NET, Unity games

2011
Kotlin
Android, runs on the JVM

2014
Swift
iOS apps

Learn OOP once. Switching languages later is mostly new syntax.

PART 2 OF 4

Java

Install it, then compile and run a program without any editor.

In Java, all code belongs to a class.

Methods, fields, even lambdas and streams sit inside classes.

02 · JAVA

Java Characteristics

- **Simple**
- **Object-Oriented**
- **Distributed**
- **Interpreted**
- **Robust**
- **Secure**
- **Architecture-Neutral**
- **Portable**
- **Performant**
- **Multithreaded**
- **Dynamic**

From the original Java white paper

Characteristics of Java

Java is Simple
Java is Object-Oriented
Java is Distributed
Java is Interpreted
Java is Robust
Java is Secure
Java is Architecture-Neutral
Java is Portable
Java's Performance
Java is Multithreaded
Java is Dynamic

Java is Simple

Java is partially modeled on C++, but greatly simplified and improved. Some people call it "C++--": like C++, with more functionality and fewer negative aspects.

- Shares many syntax elements with C/C++
- Learning curve less steep for C/C++ programmers

02 · JAVA

Characteristics of Java

- Java is Simple
- **Java is Object-Oriented**
- Java is Distributed
- Java is Interpreted
- Java is Robust
- Java is Secure
- Java is Architecture-Neutral
- Java is Portable
- Java's Performance
- Java is Multithreaded
- Java is Dynamic

Java is Object-Oriented

Java was designed from the start to be object-oriented. OOP gives flexibility, modularity, clarity and reusability through encapsulation, inheritance and polymorphism.

- Quasi-pure OO: only classes and objects, no free functions or pointers
- Basic types (int, double…) are the exception

02 · JAVA

Characteristics of Java

- Java is Simple
- Java is Object-Oriented
- **Java is Distributed**
- Java is Interpreted
- Java is Robust
- Java is Secure
- Java is Architecture-Neutral
- Java is Portable
- Java's Performance
- Java is Multithreaded
- Java is Dynamic

Java is Distributed

Distributed computing means several computers working together on a network. Networking is built into Java, so writing network programs is like sending and receiving data to and from a file.

- Network programming in the standard library: sockets, RMI

02 · JAVA

Characteristics of Java

- Java is Simple
- Java is Object-Oriented
- Java is Distributed
- **Java is Interpreted**
- Java is Robust
- Java is Secure
- Java is Architecture-Neutral
- Java is Portable
- Java's Performance
- Java is Multithreaded
- Java is Dynamic

Java is Interpreted

Programs are compiled into Java Virtual Machine code called bytecode. Bytecode is **machine-independent** and runs on any machine that has a JVM.

- Translated to an intermediate language: bytecode
- Interpreted with optimizations (JIT)

02 · JAVA

Characteristics of Java

- Java is Simple
- Java is Object-Oriented
- Java is Distributed
- Java is Interpreted
- **Java is Robust**
- Java is Secure
- Java is Architecture-Neutral
- Java is Portable
- Java's Performance
- Java is Multithreaded
- Java is Dynamic

Java is Robust

Java compilers detect many problems that would only show up at run time in other languages. Error-prone constructs from other languages are gone, and runtime exception handling supports robustness.

- Strong type model, no explicit pointers: compile-time checks
- Run-time checks: no array overflow
- Garbage collection: no manual memory freeing
- Exceptions: a pervasive mechanism to check errors

02 · JAVA

Characteristics of Java

- Java is Simple
- Java is Object-Oriented
- Java is Distributed
- Java is Interpreted
- Java is Robust
- **Java is Secure**
- Java is Architecture-Neutral
- Java is Portable
- Java's Performance
- Java is Multithreaded
- Java is Dynamic

Java is Secure

Java implements several security mechanisms to protect your system against harm caused by stray programs.

- Bytecode is verified before it runs

02 · JAVA

Characteristics of Java

- Java is Simple
- Java is Object-Oriented
- Java is Distributed
- Java is Interpreted
- Java is Robust
- Java is Secure
- Java is Architecture-Neutral
- Java is Portable
- Java's Performance
- Java is Multithreaded
- Java is Dynamic

Java is Portable

Because Java is architecture-neutral, Java programs are portable: they run on any platform without being recompiled.

- Lots of standard utilities included: threads, GUI, networking

02 · JAVA

Characteristics of Java

- Java is Simple
- Java is Object-Oriented
- Java is Distributed
- Java is Interpreted
- Java is Robust
- Java is Secure
- Java is Architecture-Neutral
- Java is Portable
- **Java's Performance**
- Java is Multithreaded
- Java is Dynamic

Java's performance

The JVM's just-in-time (JIT) compiler turns frequently executed bytecode into native machine code while the program runs, so Java programs run fast.

- Bytecode + JIT: close to native speed

Characteristics of Java

Java is Simple
Java is Object-Oriented
Java is Distributed
Java is Interpreted
Java is Robust
Java is Secure
Java is Architecture-Neutral
Java is Portable
Java's Performance
Java is Multithreaded
Java is Dynamic

Java is Multithreaded

Multithreaded programming is smoothly integrated in Java. In other languages you call procedures specific to the operating system to get threads.

- Concurrency (threads) is part of the standard library

02 · JAVA

Characteristics of Java

- Java is Simple
- Java is Object-Oriented
- Java is Distributed
- Java is Interpreted
- Java is Robust
- Java is Secure
- Java is Architecture-Neutral
- Java is Portable
- Java's Performance
- Java is Multithreaded
- **Java is Dynamic**

Java is Dynamic

Java adapts to an evolving environment: new code can be loaded on the fly, without recompiling, and new features incorporated transparently as needed.

- Run-time loading and linking
- Dynamic array sizes

02 · JAVA

Java, web and beyond

Standalone applications

Desktop programs and tools, like Eclipse itself.

Web servers

The back-end of banking, shopping and booking sites.

Hand-held devices

Android apps run Java and Kotlin.

From a browser

Once via applets; today the browser talks to a Java server.

02 · JAVA

One first-level concept: the class

- There is only one first-level concept: the **class**

First.java

```
public class First {
}
```

- The source code of a class sits in a *.java* file with the **same name**
  - Rule: one file per class
  - Enforced automatically by IDEs
  - Case-wise name correspondence: first.java ≠ First.java

02 · JAVA

No functions, only methods

- In Java there are no functions, only methods within classes
- Execution starts from one special method:

```
public static void main(String[] args)
```

  - Return type is **void**. In C: **int main(int argc, char* argv[])**
  - **args[0]** is the first argument on the command line, after the program name

02 · JAVA

Example: source code

First.java

```java
public class First {
  public static void main(String[] args) {
    int a;
    a = 3;
    System.out.println(a);
  }
}
```

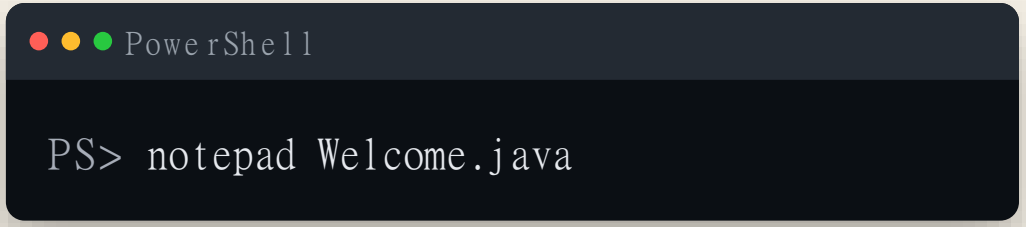
What does it print?

Then: javac First.java, then java First.

02 · JAVA

Creating and editing using Notepad

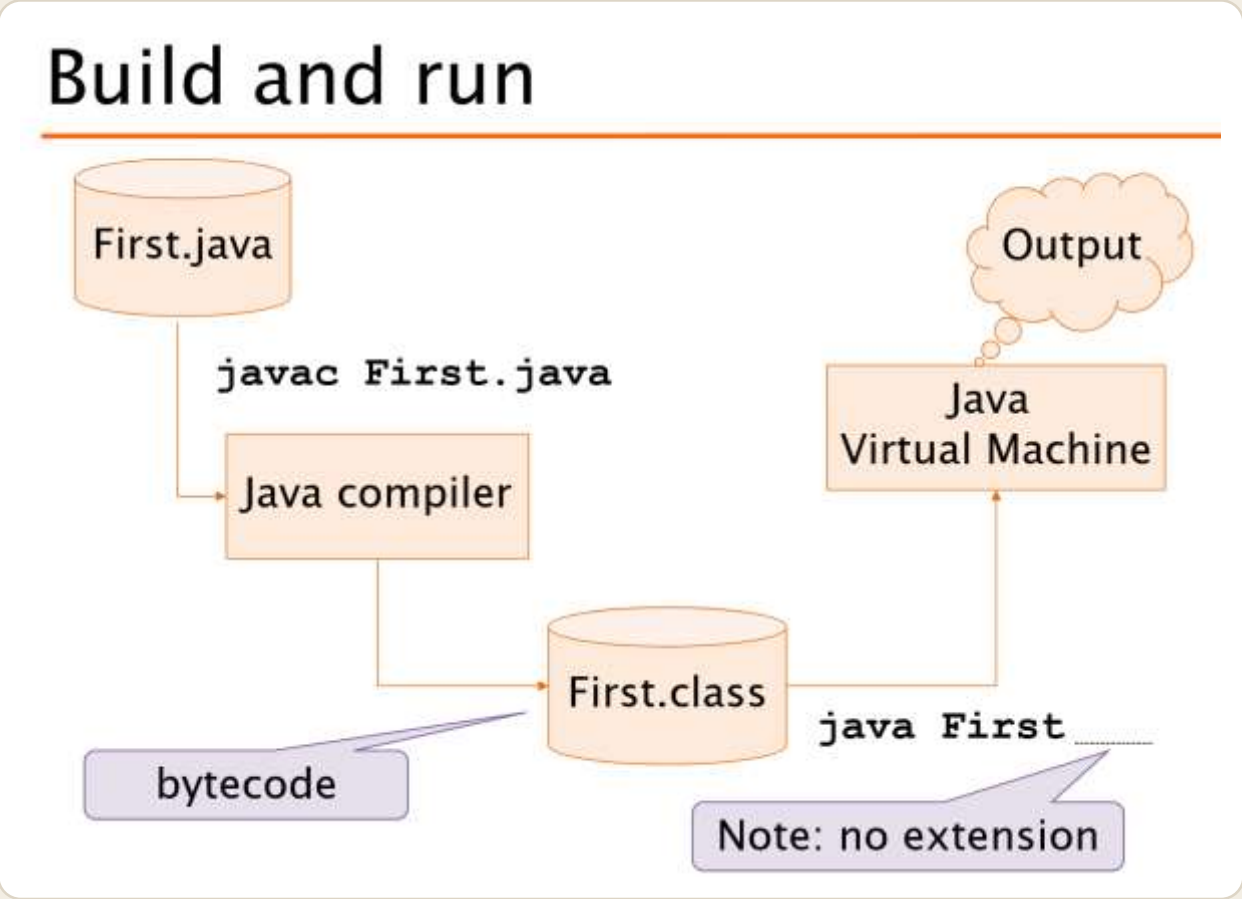
To use Notepad, type **notepad Welcome.java** at the prompt.



Welcome.java - Notepad

```
// This application program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

Build and run

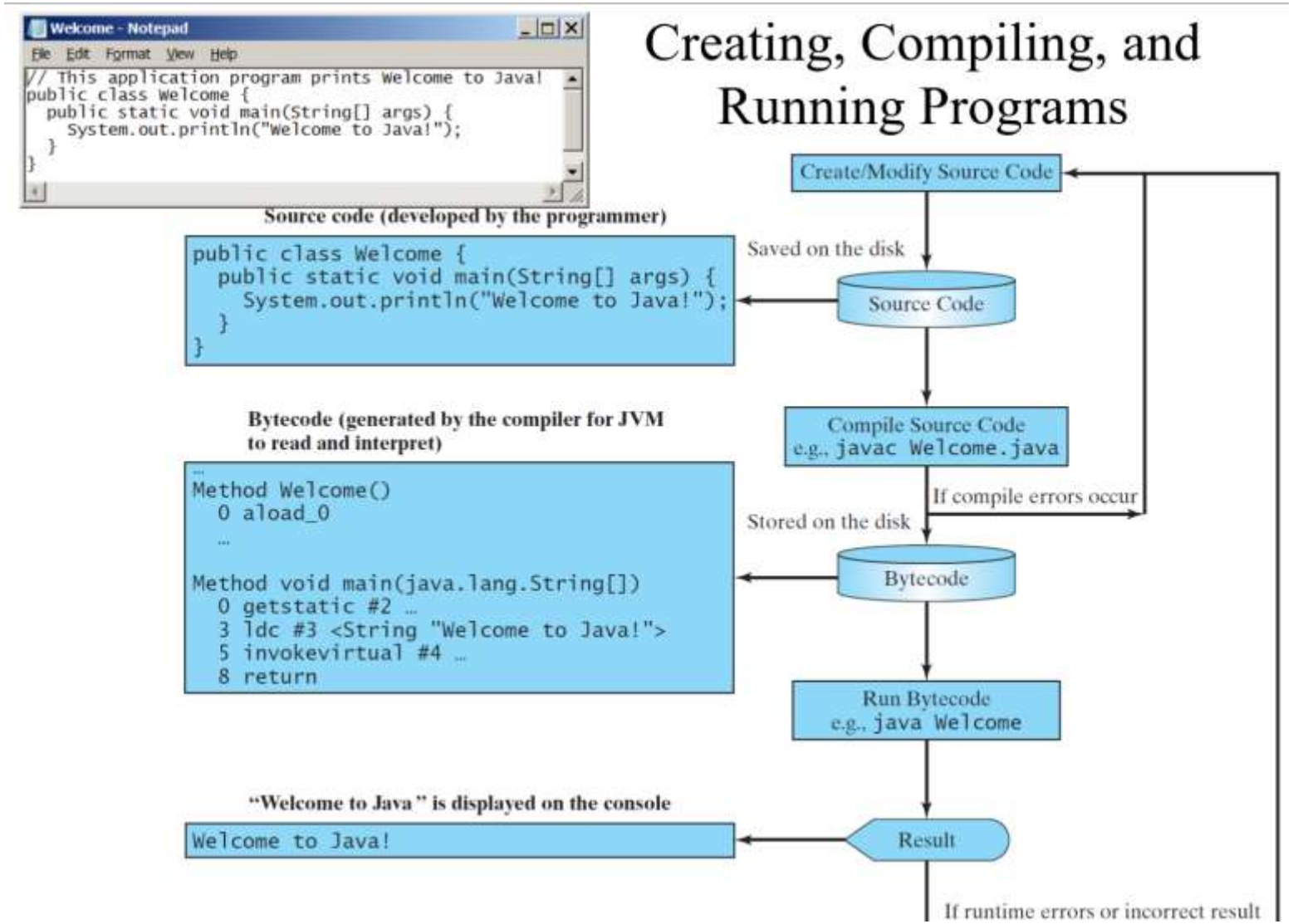


javac First.java turns source into **First.class**: bytecode.

java First runs it on the JVM.

No extension!

Creating, Compiling, and Running Programs



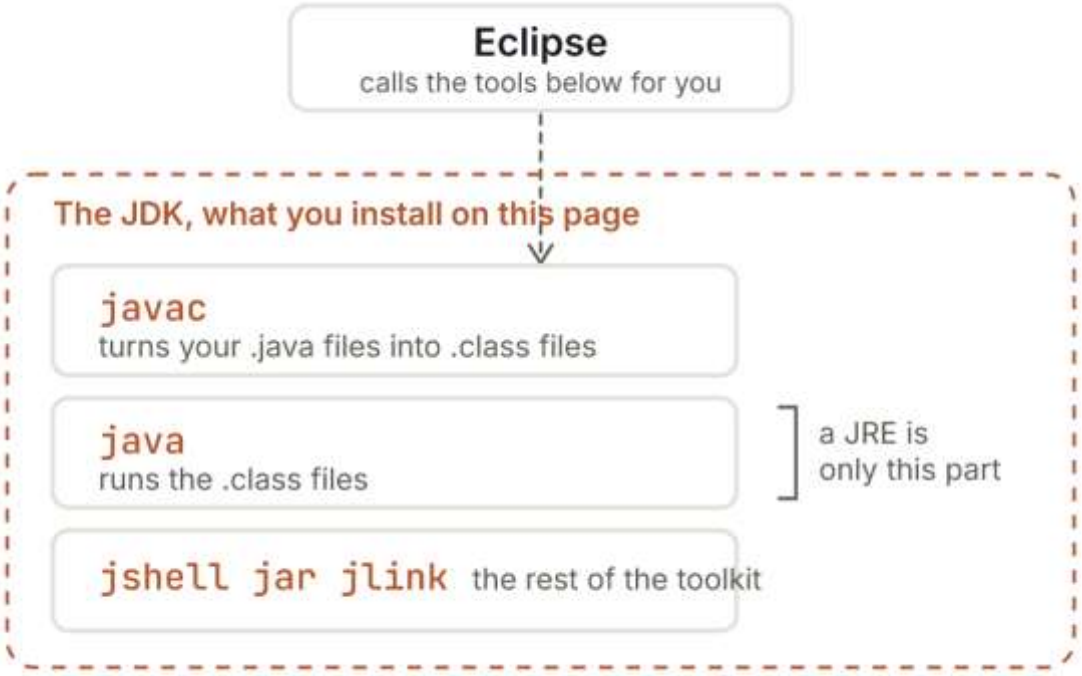
02 · JAVA

What happens when you type java First

- Command: **java First**
  - Take the name of the class (**First**)
  - Look for the bytecode for that class, in the classpath (and the current folder)
  - Load the class's bytecode and perform all due initializations
  - Look for the **main()** method
  - Start execution from the **main()** method

02 · JAVA

What you actually install: the JDK



A JRE can run Java. It cannot compile it.

Install a JRE by mistake and you can run other people's programs but not your own. We install the full JDK.

02 · JAVA

Which Java? This course: 25

A new Java every six months. Every two years one is **LTS**, long-term support. Companies stay on LTS versions for years.

History: JDK 1.0 (1996), 1.2 (1998), 5 (2004), 6 (2006), 7 (2011), 8 (2014). Since 2017, every six months.

8	11	17	21	**25**
2014	2018	2021	2023	2025

We use Eclipse Temurin 25, the free OpenJDK build. Not Oracle's download: different licence, same Java.



DO IT NOW · FOLLOW ALONG

Install Java 25 (Temurin JDK)

1. Open **PowerShell**: Start menu, type PowerShell.
2. Run the install command on the right.
3. **Close PowerShell and open a new one.** PATH only updates in new windows.
4. No winget? Use the .msi from adoptium.net: Windows, x64, JDK, 25. Tick **Set JAVA_HOME** and **Add to PATH**.

PowerShell

```
PS> winget install -e --id
    EclipseAdoptium.Temurin.25.JDK
```

Mac: download the .pkg from adoptium.net and double-click it. Apple chip (M1–M4): **aarch64**. Intel: **x64**.

adoptium.net ↗ | Course page: Install Java ↗

DO IT NOW · FOLLOW ALONG

Prove it: no editor needed

1 · Check the versions

```
PS> java -version
openjdk version "25.0.x" ...
PS> javac -version
javac 25.0.x
PS> jshell
jshell> System.out.println("Running on "
        + Runtime.version());
Running on 25.0.x
jshell> /exit
```

2 · Compile by hand

```
PS> mkdir C:\dev\jdkcheck
PS> cd C:\dev\jdkcheck
PS> notepad Check.java
...type the class, save...
PS> javac Check.java
PS> java Check
Compiled and running on 25.0.x
```

Check.java prints "Compiled and running on " + Runtime.version(). Both numbers must start with 25. Mac: use Terminal and ~/dev/jdkcheck.

02 · JAVA

Reading the program

Check.java

```java
public class Check {
    public static void main(String[] args) {
        System.out.println("Hello, OOP!");
    }
}
```

Console

```
Hello, OOP!
```

1. **`public class Check`**
   File must be named Check.java
2. **`main(...)`**
   Where every program starts
3. **`System.out.println`**
   Prints one line to the console
4. **`;`**
   Ends every statement. Don't forget it.

EXPECTATION

It just works.

PowerShell

```
PS> javac Check.java
PS> java Check
Hello, OOP!
```

REALITY

One tiny semicolon.

PowerShell

```
Check.java:3: error: ';' expected
        System.out.println("Hello, OOP!")
                                          ^
1 error
```

02 · JAVA

Java cares about names. A lot.

PowerShell

```
PS> javac check.java
check.java:1: error: class Check is public, should be
declared in a file named Check.java
public class Check {
       ^
1 error
```

⚠ ~~check.java~~ ⚠ ~~Check.java.txt~~ ✓ Check.java

Notepad loves adding .txt. In File Explorer turn on View › Show › File name extensions.

02 · JAVA

“Java can’t find my class”

PowerShell

```
Error: Could not find or load main class First
Caused by: java.lang.ClassNotFoundException: First
```

- Don’t include the extension: **java First**, not java First.class
- Check you are in the right folder: java looks for classes starting from the current working directory
- Check the capitals: first is not First

02 · JAVA

Coding conventions

- Use **camelBackCapitalization** for compound names, not underscores
- Class names must be **C**apitalized
- Method names, object names, attributes and variables start in lowercase
- Constants are all UPPERCASE (with underscores)
- Indent properly

02 · JAVA

Coding conventions: example

ClassName.java

```
class ClassName {

static final double PI = 3.14;
private int attributeName;

  public void methodName() {
    int count = 0;
     if (count == 0) { }
  }
}
```

02 · JAVA

Deployment: jar files

- Java programs are packaged and deployed in jar files
- A jar is a compressed archive
  - Like a zip file
  - Contains additional meta-information
- A JVM can run the contents of a jar directly
  - The JVM loads classes from inside the jar: **java -jar app.jar**

02 · JAVA

Java in one slide

- Java is a quasi-pure object-oriented language
- Compiled to bytecode, then interpreted with JIT
- Robust: no explicit pointers, static and dynamic checks, garbage collection
- Many utilities: data types, threads, networking, graphics
- Used for many kinds of programs
- Coding conventions are not “just aesthetic”

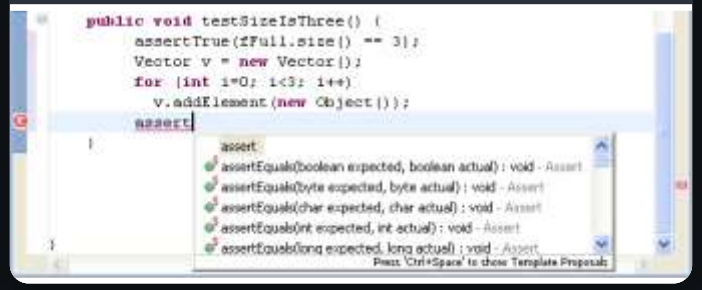
PART 3 OF 4

Eclipse

03 · ECLIPSE

Shortcuts worth learning today

Autocomplete

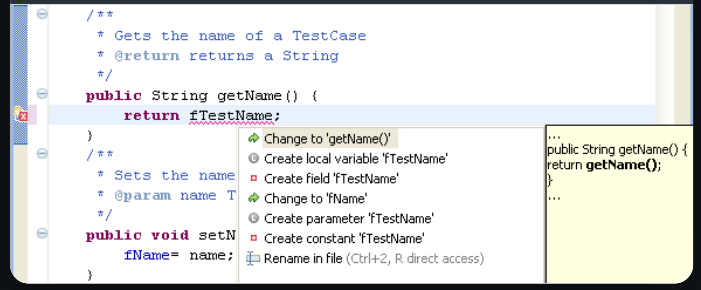


Autocomplete

Windows `Ctrl + Space`

Mac `Ctrl + Space`

Quick fix

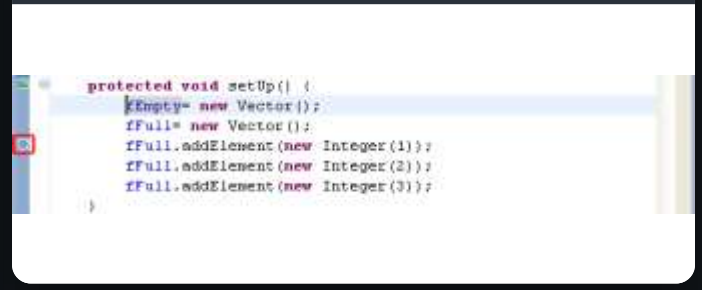


Quick fix

Windows `Ctrl + 1`

Mac `⌘ Cmd + 1`

Debugger



Debugger

Windows `F11`

Mac `⌘ Cmd + F11`

Run
Win Ctrl+F11
Mac ⌘+Shift+F11

Format
Win Ctrl+Shift+F
Mac ⌘+Shift+F

Rename
Win Alt+Shift+R
Mac ⌥+⌘+R

Open class
Win Ctrl+Shift+T
Mac ⌘+Shift+T

03 · ECLIPSE

All good tools. We pick one.

IDE	Cost	Strengths	Keep in mind
Eclipse	Free, open source	Mature Java tools, plugins	Interface feels busy at first
IntelliJ IDEA	Free tier + paid edition	Excellent code assistance	Heavier on memory
VS Code	Free	Lightweight, many languages	Java support via extensions
NetBeans	Free, open source	Simple setup, GUI builder	Smaller community
BlueJ	Free	Designed for teaching OOP	Not used in industry

Prefer another editor? Fine: projects are plain Java (later plain Maven). You still need Java 25 and git.

03 · ECLIPSE

Why we use Eclipse

Same screens for everyone.

A problem takes **two minutes** to solve instead of twenty.

- **Built for Java**
  Errors and quick fixes as you type.
- **Strong debugger**
  Breakpoints, variables, step by step.
- **Free & open source**
  Same tool at home and in the lab.
- **Nothing depends on it**
  Plain Java projects you can build anywhere.

DO IT NOW · FOLLOW ALONG

Install Eclipse

1. eclipse.org/downloads › **Eclipse Installer**, Windows x86_64.
2. Check the file before running it: compare its **SHA-512** with the one on the download page.
3. Run it › **Eclipse IDE for Java Developers** › Install › Launch. Blue SmartScreen box? More info › Run anyway (only after the hash matched).
4. Workspace: C:\ dev\ workspace. Tick **Use this as the default**.

PowerShell · step 2

```
PS> Get-FileHash
  "$HOME\Downloads\eclipse-inst-jre-win64.exe"
  -Algorithm SHA512
```

Mac: Installer for **AArch64** (Apple chip) or x86_64 (Intel). Hash: shasum -a 512 file.dmg. Workspace: ~/dev/workspace

eclipse.org/downloads ↗

Course page: Eclipse ↗

DO IT NOW · FOLLOW ALONG

Point Eclipse at your JDK 25

1. **Window › Preferences › Java › Installed JREs**
2. See **jdk-25** listed and ticked? Done. Eclipse found it.
3. Otherwise: **Add › Standard VM › Next**. JRE home: C:\Program Files\Eclipse Adoptium\jdk-25…
4. Finish, **tick the new entry**, Apply and Close.

Eclipse ships with its own small Java. Without this step it can **seem** to work, then fail later in a way that looks unrelated.

Mac: Eclipse › Settings… › Java › Installed JREs. JDK folder: /Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home

DO IT NOW · FOLLOW ALONG

Build something, then run it without Eclipse

1. **File › New › Java Project**, name it **Setup**, Finish. (If "Create module-info.java" is ticked, untick it.)
2. Right-click **src › New › Class**, name it **Check**, tick **public static void main**, Finish.
3. Paste the line on the right inside main.
4. Run: **Ctrl+F11** or the green ▶. Console says **Eclipse is running on 25…**

inside main

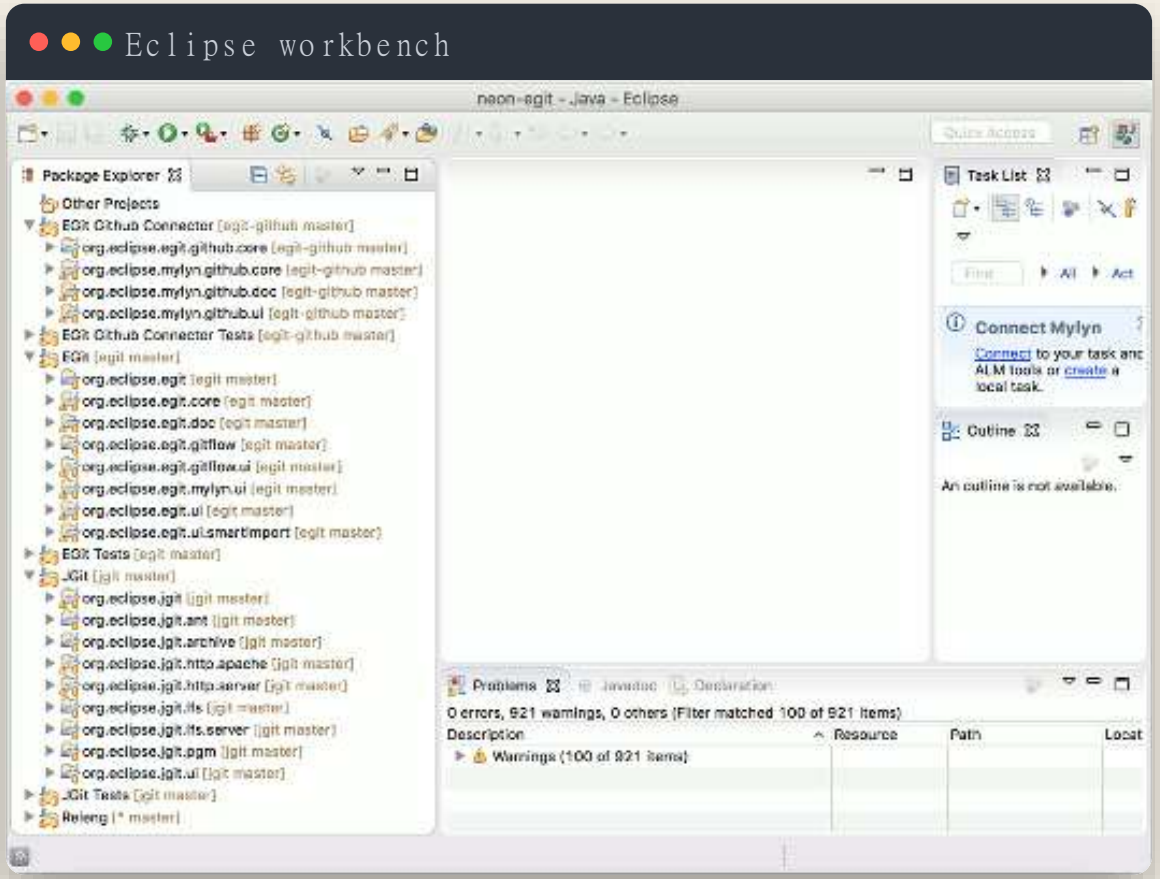
```
System.out.println(
"Eclipse is running on "
+ Runtime.version());
```

Then, outside Eclipse

```
PS> cd C:\dev\workspace\Setup\bin
PS> java Check
Eclipse is running on 25.0.x
```

03 · ECLIPSE

Finding your way around Eclipse



1. **Package Explorer**
   Projects and files (left)
2. **Editor**
   Where you write code (centre)
3. **Outline**
   Fields and methods of the class (right)
4. **Problems · Console**
   Errors and program output (bottom)

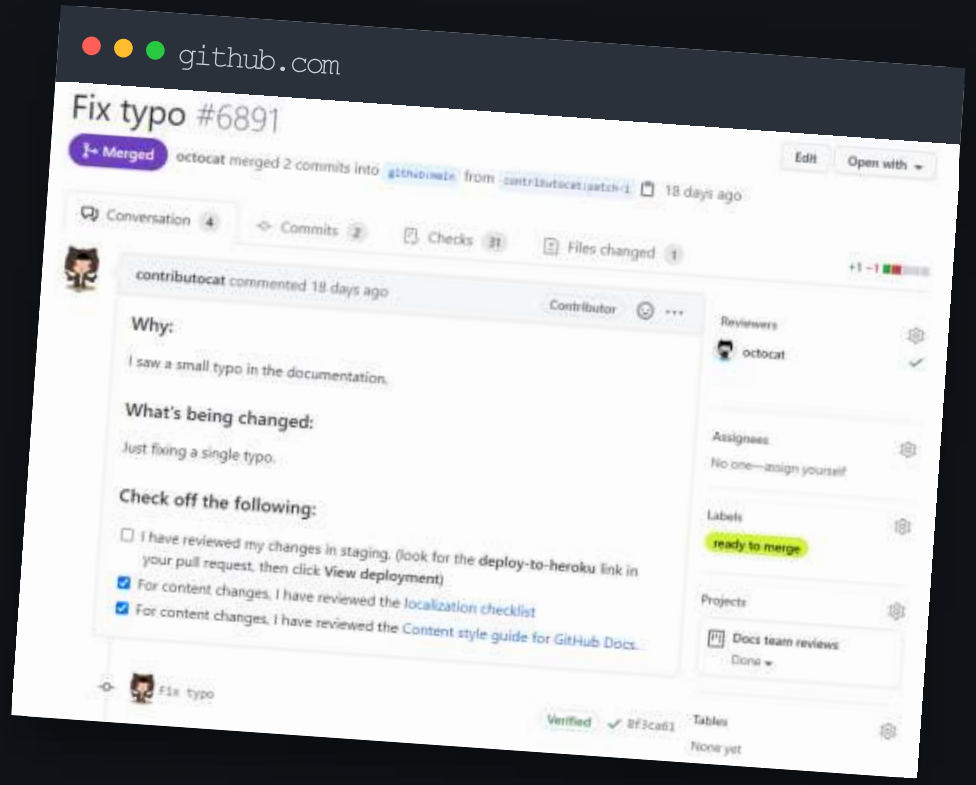
Turn on line numbers: Preferences › General › Editors › Text Editors › Show line numbers.

Panel gone? Window › Perspective › Reset Perspective.

PART 4 OF 4

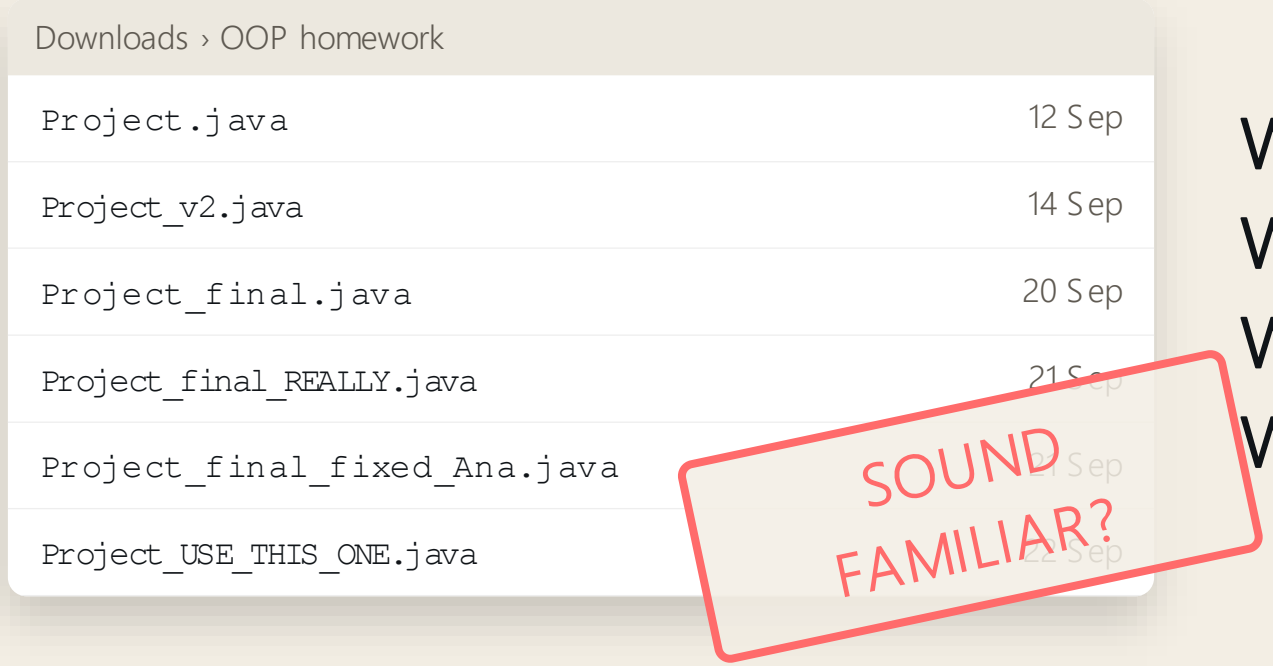
Git & GitHub

Your work lives in a repository from week one, not in a folder on your laptop.



04 · GIT & GITHUB

Be honest. Does your folder look like this?



Who changed what?

When?

Why?

Which one works?

04 · GIT & GITHUB

Git and GitHub are not the same thing

Git

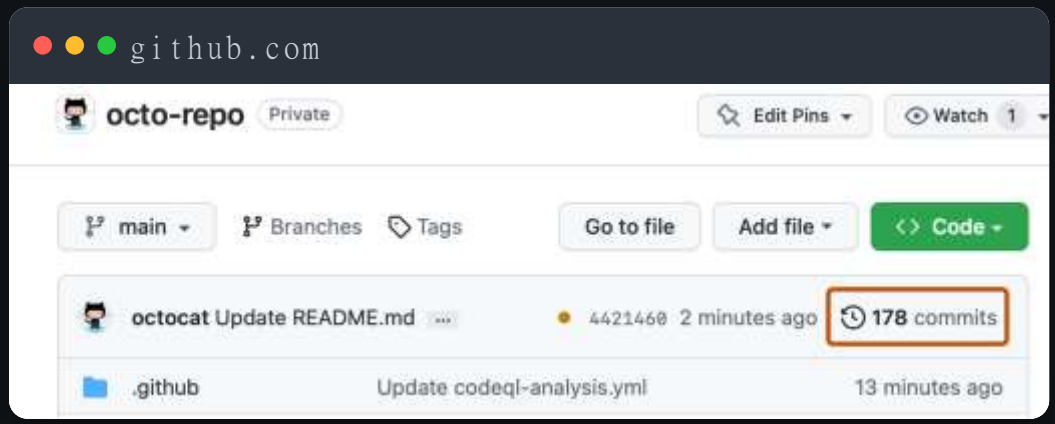
A tool on **your computer**. Works offline, no account. Created by Linus Torvalds in 2005.

Terminal · your laptop

```
$ git log --oneline
97b6b9b Add .gitignore for bin/ and .metadata/
e261f1f Fix rounding in getAverage()
da184f5 Add getAverage() to Student
e81e11f Add Student class with name and grade
51ed780 Add HelloWorld program
```

GitHub

A **website** that hosts Git repositories, plus pull requests, issues, reviews.



Git is the tool. GitHub is where you share what Git tracks.

04 · GIT & GITHUB

Six words, explained like a video game

Your save file

Repository

A project folder whose whole history Git tracks.

A save point

Commit

A snapshot with a message, author and time.

An alternate route

Branch

A parallel line of work. Default: main.

Routes rejoin

Merge

Combine one branch's changes into another.

Cloud save

Remote

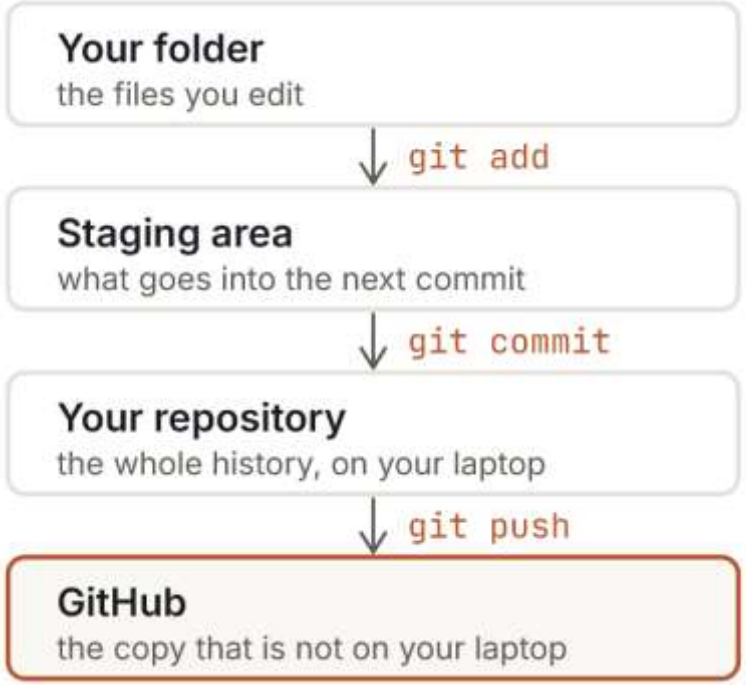
The copy on GitHub, usually called origin.

Copy the save

Clone

Download a repo with its full history.

The four places your work lives



Three of the four are on your laptop.

Only **git push** needs the internet. That's why most of git keeps working on a train.

git add chooses what goes in the next snapshot. **git commit** takes it. **git push** copies it to GitHub.

DO IT NOW · FOLLOW ALONG

Install Git and introduce yourself

1. Install Git with the command on the right. Accept every default: they include **Git Credential Manager**.
2. Close PowerShell, open a new one, run **git --version**.
3. Tell git who you are, once per computer. Use the email you'll use on GitHub.
4. Create a free account on **github.com** with that email.

PowerShell

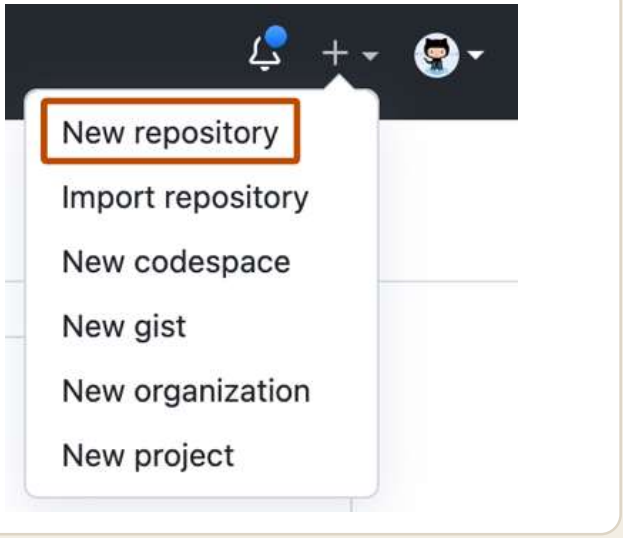
```
PS> winget install -e --id Git.Git
PS> git --version
PS> git config --global user.name "Ana Hoxha"
PS> git config --global user.email "ana@example.com"
```

Mac: run git --version in Terminal. If asked, click **Install** (Command Line Tools). That gives you git.

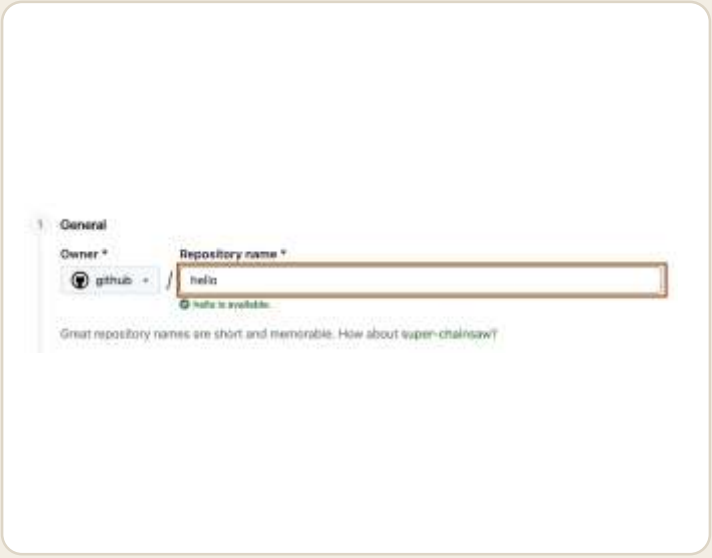
git-scm.com ↗ Course page: Git ↗

Create your repository on GitHub

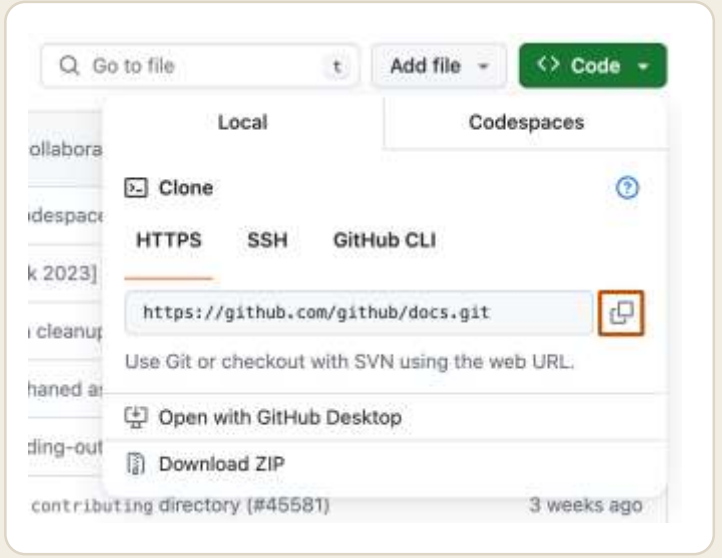
1 **+ › New repository**



2 **Name: java-oop-surname**



3 **Copy the HTTPS URL**



On the next page, under **Quick setup**, make sure **HTTPS** is selected and copy the URL. Ignore the commands below it.
Do not tick Add a README.

DO IT NOW · FOLLOW ALONG

Turn your project into a repository

PowerShell

```
PS> cd C:\dev\workspace\Setup
PS> git init
PS> notepad .gitignore
```

.gitignore

```
bin/
*.class
.settings/
.classpath
.project
```

Never commit bin/. It is rebuilt from your code in a second and changes on every build.

Commit what you wrote, never what a tool produced from it.

Mac: cd ~/dev/workspace/Setup, then create the file with: open -e .gitignore (after touch .gitignore)

DO IT NOW · FOLLOW ALONG

Your first commit

PowerShell

```
PS> git add .
PS> git status
Changes to be committed:
        new file: .gitignore
        new file: src/Check.java
PS> git commit -m "First Java project, verifying the setup"
```

Read git status before you commit.

See bin/ in the list? Your .gitignore is in the wrong folder or has a typo.

DO IT NOW · FOLLOW ALONG

Connect to GitHub and push

PowerShell

```
PS> git branch -M main
PS> git remote add origin
  https://github.com/yourname/java-oop-surname.git
PS> git push -u origin main
```

A **browser window opens** asking you to sign in to GitHub. That's Git Credential Manager. Approve once; it won't ask again on this machine.

Refresh your repository page: **your files are there.**

Mac: Terminal asks for a username and password instead. The password is a **personal access token** (GitHub › Settings › Developer settings), not your GitHub password.

04 · GIT & GITHUB

git status

Before git add

```
PS> git status
On branch main
No commits yet
Untracked files:
  (use "git add <file>..." to include)
        Check.java
```

After git add

```
PS> git add Check.java
PS> git status
On branch main
Changes to be committed:
        new file:   Check.java
```

Red = Git sees it but won't save it. Green = in the next commit.

04 · GIT & GITHUB

The cheat sheet: eight commands

Command	What it does
git config --global user.name "Ana"	Set your name once per computer (also user.email)
git init	Turn a folder into a repository
git status	See what changed and what is staged
git add <file> / git add .	Stage changes for the next commit
git commit -m "message"	Save a snapshot locally
git push	Upload your commits to GitHub
git pull	Download and merge new commits
git log --oneline	Show the history

04 · GIT & GITHUB

git log never forgets

DON'T

Terminal

```
$ git log --oneline
f85b523 please work
bb5ccde final version 2
af40477 final version
b58b6d5 asdf
3a13266 fixed
1a4fc2a stuff
```

DO

Terminal

```
$ git log --oneline
97b6b9b Add .gitignore for bin/ and .metadata/
e261f1f Fix rounding in getAverage()
da184f5 Add getAverage() to Student
e81e11f Add Student class with name and grade
51ed780 Add HelloWorld program
```

A good message finishes the sentence: “If applied, this commit will…”

04 · GIT & GITHUB

Rejected? Don't panic.

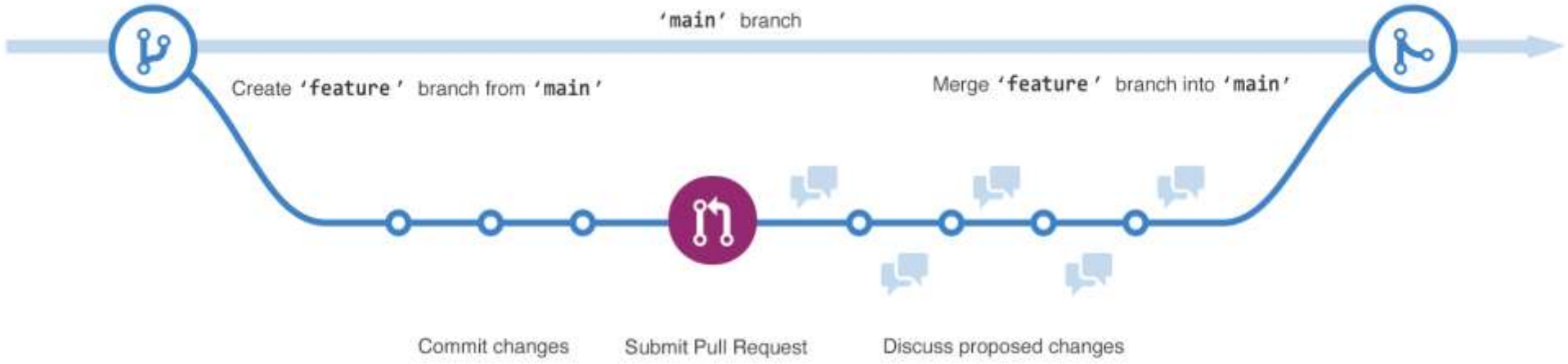
PowerShell

```
PS> git push -u origin main
! [rejected]        main -> main (fetch first)
error: failed to push some refs
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. ... use 'git pull' before pushing again.
```

The fix: `git pull --rebase origin main` → `git push`

04 · GIT & GITHUB

Later, in teams: branch, then pull request

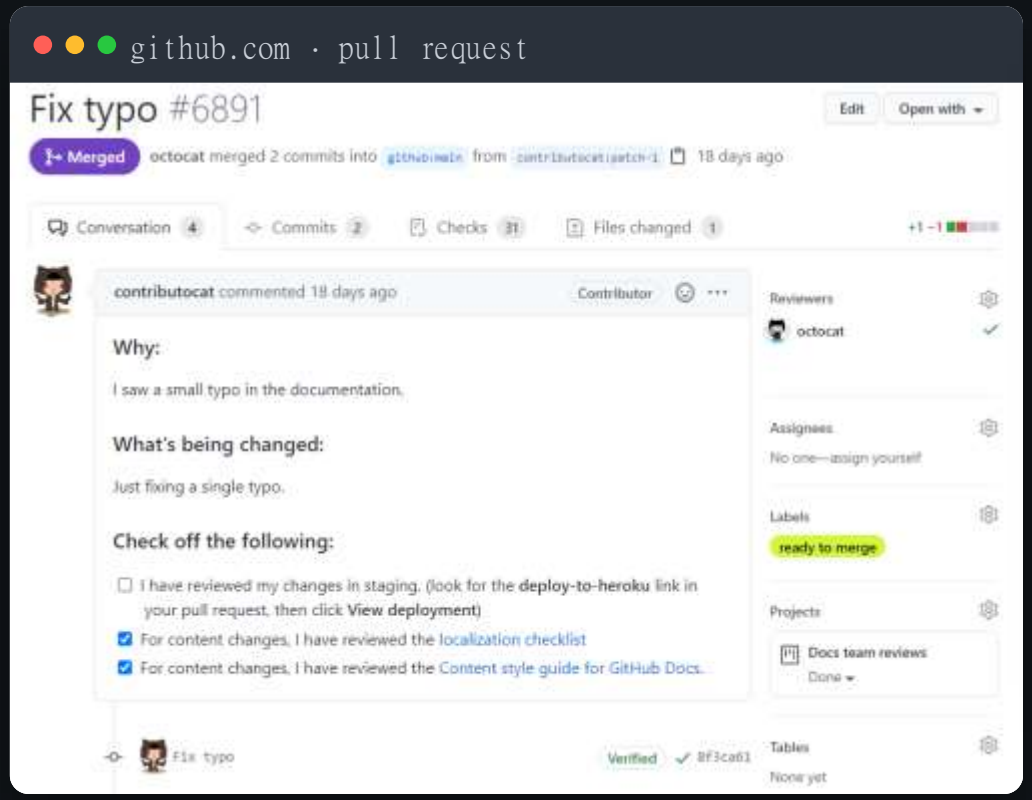


1 Branch | 2 Commit | 3 Push | 4 Open PR | 5 Review | 6 Merge

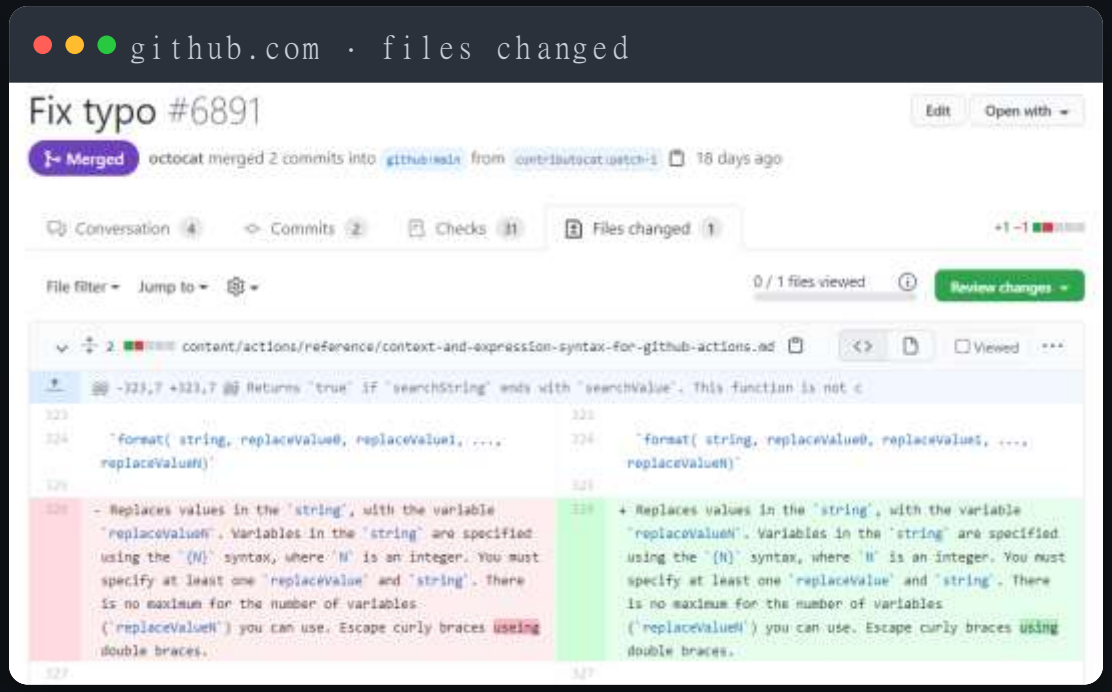
04 · GIT & GITHUB

What a pull request looks like

CONVERSATION: describe and discuss

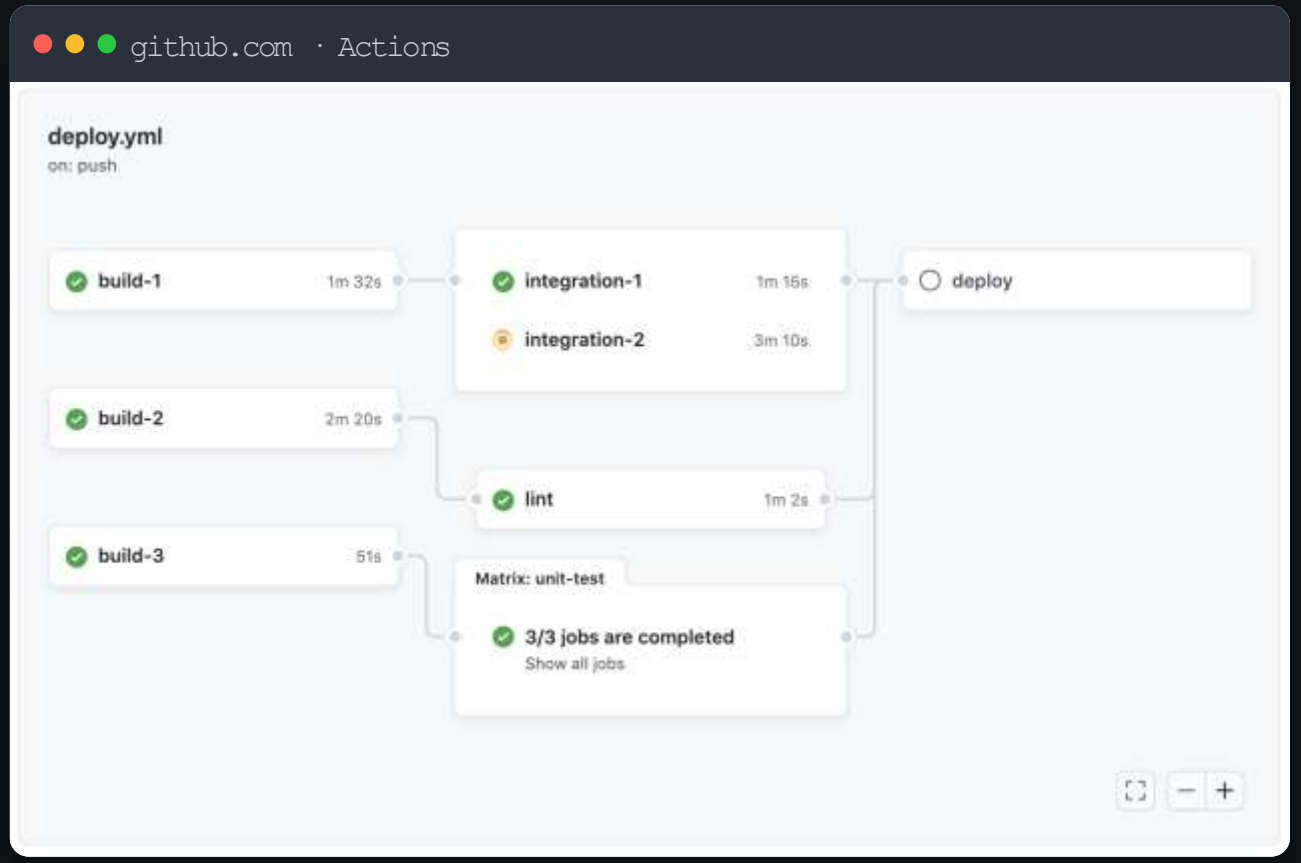


FILES CHANGED: review line by line



04 · GIT & GITHUB

A quick tour of GitHub



README.md
The front page, in Markdown

Commits
Your history, with names and messages

Issues
Bugs, tasks and questions

Pull requests
Propose and review changes

Actions
Build and test on every push

.gitignore
Files Git must never track

04 · GIT & GITHUB

Six Git habits for this course

01

Commit every seminar

Small, focused commits, at least one per session.

02

Explain the change

"Add academicStanding()", not "stuff".

03

Pull first, push last

Start with pull; end with push.

04

No secrets

Never commit passwords or keys.

05

Ignore build output

bin/ stays out, always.

06

History is your portfolio

By week 14 it shows how you built it.

WHEN THINGS BREAK

Read the message. Find it. Then ask.

Each course setup page has a troubleshooting section with the exact errors you're likely to see.

TROUBLESHOOTING

Java

You see	Fix
'java' is not recognized…	Open a new terminal. Still failing? Re-run the MSI › Change › turn on Set JAVA_HOME and Add to PATH. Check with where.exe java.
The version is not 25	An older Java is found first. where.exe java and where.exe javac; remove extra ones in Settings › Apps.
java works, javac doesn't	You installed a JRE, not a JDK. Install the package ending in .JDK.
winget: package not found	winget source update, then try again, or use the MSI.
Could not find or load main class	Run java Check (no .class) from the folder that holds Check.class.

TROUBLESHOOTING

Eclipse

You see	Fix
No Java virtual machine was found	The problem isn't Eclipse: java -version in a fresh terminal. Fix Java first.
UnsupportedClassVersionError	Project › Properties: Java Build Path › Libraries and Java Compiler must both say 25.
One project misbehaves, others fine	Properties › Java Compiler › "Enable project specific settings" outranks the workspace.
Random errors, files reverting	Workspace is in a synced folder. Move it to C:₩dev₩workspace; File › Switch Workspace.
Eclipse is extremely slow	Antivirus scanning every file. Exclude the workspace and Eclipse folders.

TROUBLESHOOTING

Git and GitHub

You see	Fix
Password authentication is not supported	Use Git Credential Manager: where.exe git must be under C:\Program Files\Git\. Then git config --global credential.helper manager.
src refspec main does not match any	Nothing committed yet. git status, then finish the first commit.
Updates were rejected	git pull --rebase origin main, then git push -u origin main.
You committed bin/	git rm -r --cached bin, commit, push. Files stay on disk.
fatal: not a git repository	Wrong folder or no git init. Check with pwd.

TROUBLESHOOTING

Mac users: the same steps, different details

Java	Download the .pkg from adoptium.net: aarch64 for Apple chips (M1–M4), x64 for Intel. Check with java -version in Terminal.
Folders	~/dev/workspace instead of C:₩dev₩workspace.
Eclipse	Installer for AArch64 or x86_64. First launch: right-click › Open. Preferences are under Eclipse › Settings…
Shortcuts	Cmd instead of Ctrl, except Ctrl+Space. Run: Cmd+Shift+F11. Quick fix: Cmd+1.
Git	Type git --version and click Install (Command Line Tools).
Push	The password prompt wants a personal access token (GitHub › Settings › Developer settings), not your password.

BEFORE YOU LEAVE

Check all five

☐ java -version and javac -version	both report 25
☐ Eclipse opens and runs Check	Console shows "running on 25"
☐ Your repo java-oop-surname exists	on GitHub, with your name on it
☐ The whole loop works	add a second println, commit, push, see it on GitHub
☐ Two commits in the history	click the commits link to check

Questions?